

Keeping delivery promises at Olist

IT5006 Phase 2 — a two-stage delivery-time estimate (regression) and a late warning at carrier handover (classification)

Code: https://github.com/YuanJiayi/Team6_IT5006_Ecommerce_Analytics_AY2627Sem1

Executive summary

Olist’s delivery operations team needs to know, for every order, how long it will take and which orders need attention. We answer this one question twice, at two points operations actually acts: at **checkout**, to set the customer’s expected date, and again at **carrier handover**, when the parcel leaves the seller, to sharpen that estimate and flag orders at risk of missing it. Both decisions concern one quantity, delivery time (brief Example 4’s dual framing), so we reuse the same two model families – Linear and Tree-based – for both.

- **Two-stage delivery-time estimate (regression).** Ridge regression ($\alpha = 100$) predicts `delivery_days` at checkout from order, route and seller details (mean validation MAE 5.81 days, R^2 0.167). The same model, refitted with what is known at handover (time already elapsed, recent route transit), cuts mean validation MAE to 5.10 days and raises R^2 to 0.338 – the handover update beats the checkout estimate in every validation window.
- **Late warning (classification).** Logistic regression ($C = 0.01$) scores each order’s risk of missing Olist’s promise once it reaches the carrier, giving operations a ranked daily list (mean validation PR-AUC 0.367, against a late rate of 4–21% across the five windows). On the test period the top 10% of the list contained 41% of all late orders; a random list contains 10%.

Delivery conditions swing, and that is the real difficulty. Normally about 4% of orders are late; in the Black Friday window it was about 12%, and in February–March 2018 about 17%. Figure 1 shows this across the full period we model. Both tools are trained to be useful through these swings: the handover update’s gain over the checkout estimate holds in every validation window, not just calm ones, and the late-warning model’s ranking lift over random selection is larger, not smaller, in the high-lateness windows (Section “Model 2”).

Decision. We recommend piloting both tools alongside Olist’s current process, with monitoring: use the checkout estimate as the customer-facing date, re-issue it at handover, and size the daily warning list to the current late rate rather than a fixed share – acting on a large fixed share was not worthwhile when the test period was unusually calm (3.5% late).

Business problem and why it matters

Delivery time dominates the customer experience. The carrier leg (handover to customer) accounts for 85% of the variance in total delivery time, averaging 9.3 of the 12.6 mean total days. Among 95,627 delivered orders with a review, 62% of late orders received a bad review (score 1 or 2), against 9% of on-time orders, a ratio of 6.7. Forecasting delivery time well enough to set and update a customer-facing estimate, and flagging orders at risk once they reach the carrier, is therefore directly tied to customer experience. Real-time delivery-time forecasting is an established problem in online retail, where a better forecast and promise policy measurably raised simulated sales [5]. Customer retention is low overall: only 3.0% of the 93,159 customers

ordered more than once, and the repeat rate after a late first order (2.5%) was close to that after an on-time one (3.0%), so we do not claim an effect on repeat purchase.

The two predictions use different information. At checkout only the order, route and seller details are known. At handover we also know how long the seller took to ship and the carrier’s recent transit speed on the route, which should sharpen the estimate and tell us how much of the promised time is already used. Since the carrier leg drives 85% of the variance, knowing the order has reached the carrier should help both tasks: the regression’s R^2 rises from 0.167 at checkout to 0.338 at handover, and the late-warning classifier’s validation PR-AUC rises from 0.213 at checkout to 0.367 at handover.

Data, cohort and validation design

We use the Olist tables supplied for the course. Of 99,441 orders, 96,272 were delivered with both an actual and an estimated delivery date; these form the modelling cohort. An order is *late* if it arrives after the promised calendar date. Both models are scored on the same population: orders whose carrier handover is after purchase and before delivery, 96,272 orders, excluding 198 without a valid handover time; training uses 75,039 orders and the test period has 19,230 orders.

Temporal split, not random. The test set is every order purchased from 26 May 2018 (19,230 orders). Training and validation use earlier orders only. Validation uses five chronological windows from 22 September 2017 to 26 March 2018, and each window is fitted only on orders purchased *and delivered* before it starts. A random split would let the model learn from orders that were still in transit when the predicted order was placed, and would hide the fact that conditions change over time – a checkout-stage classifier experiment with a shuffled split scored materially higher than the chronological one for exactly this reason. Time-ordered evaluation is standard for forecasting [1], and because the late rate itself changes over time, the temporal split is the honest test of how the tools would perform going forward. As a secondary check we also score a same-period random split (Section “Model 1”); it is not used for selection because it measures interpolation within the observed period, not forecasting.

Why the handover timestamp is legitimate. Both the handover update and the late-warning classifier are scored at the moment the carrier takes the parcel, so only information available at that moment is used: approval time, days from purchase to handover, remaining slack (the promise minus days already used), and route transit history built only from deliveries already completed by that moment. Delivery and review data are never used as inputs.

Leakage and reproducibility. All preprocessing (imputation, encoding, scaling) is fitted inside each cross-validation fold’s own Pipeline, on the fitting rows only, so no information from a validation or test row ever reaches training – the same discipline applies to the route-transit and approval-time features above, which use only history available before each order’s own prediction point. All models share one fixed random seed (42) across folds and the final refit, so every result in this report is reproducible by rerunning `phase2_eta.py` and `phase2_handover_classifier.py`.

Model 1: the two-stage delivery-time estimate (regression)

Method. Both stages predict the same target, `delivery_days`, so they are directly comparable and so a single success criterion applies: the handover update must beat the checkout estimate

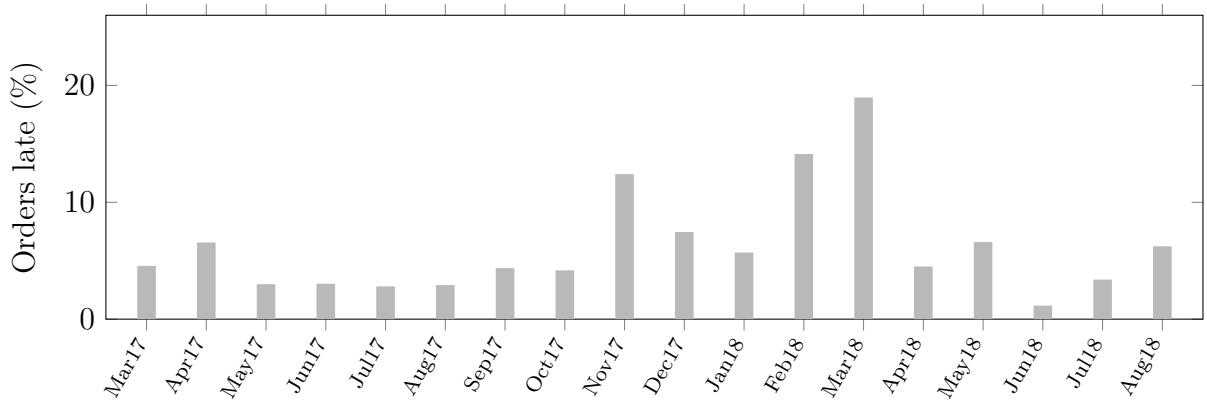


Figure 1: Share of orders late by purchase month. Black Friday (November 2017) and February–March 2018 stand out as the clearest swings in delivery conditions; the test period (June–August 2018, not shown) was calmer at 3.5% late.

on the same orders in every validation window. The **checkout** stage uses the order, route and seller features known at purchase (20 inputs). The **handover** stage adds three inputs known once the carrier has the parcel: **handover_days** (elapsed time), **approval_days** (used only when approval was recorded by handover), and **route_transit_90d_at_handover** (the route’s mean carrier-to-customer days over the previous 90 days, built only from deliveries already recorded, shrunk toward the all-route mean). Table 1 shows each input’s contribution with one fixed model (ridge, $\alpha = 100$) so the gain is not confounded with model choice: most of the improvement comes from the elapsed time itself, consistent with the carrier leg’s 85% share of delivery-time variance.

Table 1: Information ladder: ridge regression ($\alpha = 100$) throughout, same orders at every rung, mean over five validation windows.

Rung	MAE (days)	R^2	Lowest-window R^2
Checkout (20 frozen inputs)	5.81	0.167	0.060
+ recent route transit history	5.68	0.214	0.078
+ elapsed days to handover and approval	5.11	0.335	0.222
Handover (history as of handover)	5.10	0.338	0.227

Candidates and selection. At each stage we compared the same line-up: a training-mean floor, Linear (plain, then ridge at three strengths) and Tree-based (plain decision tree, then depth-6 tree, random forest and two gradient-boosting settings) – the handover stage also compares a rule baseline (elapsed days plus the route’s recent transit average, no fitted model). We select the candidate with the lowest mean validation MAE, with ties broken by the paired one-standard-error rule: the simplest candidate whose mean gap to the best, over the five validation windows, is within one standard error of that gap [3]. This is standard in statistical learning (it is `glmnet`’s default `lambda.1se`); the caveats are that we apply it across model families rather than within one penalty path (our simplicity order – linear, then tree-based – is a judgment call) and that the standard error comes from five non-independent windows.

At **checkout**, the random forest (`forest_leaf20`) has the lowest mean MAE (5.78 days), but ridge $\alpha = 100$ is within one standard error (gap 0.026 days, SE 0.028) and simpler, so it is selected (MAE 5.81, RMSE 9.40, R^2 0.167). At **handover**, ridge $\alpha = 100$ is both the lowest-MAE candidate and the one-SE selection (MAE 5.10, RMSE 8.41, R^2 0.338); no simpler candidate’s gap to it falls within one standard error. For reference, the handover rule baseline (elapsed days

plus route average, no fitted model) scores MAE 5.30 and R^2 0.337, close to the selected model: most of the signal is the elapsed time itself, and the fitted model adds a modest refinement. The training-mean floor scores MAE 7.21 (R^2 -0.078) at both stages. We also tried an equal-weight voting ensemble of each family’s best handover candidate (ridge and gradient boosting); at MAE 5.11 days it did not beat ridge alone, so ensembling was not adopted (`voting_family_winners`, Appendix A). The full candidate table is in Appendix A.

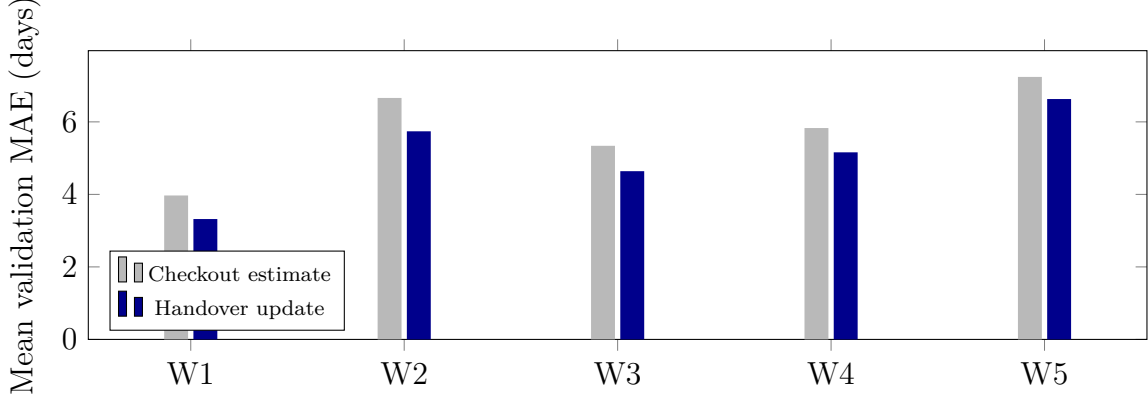


Figure 2: Mean validation MAE by window, checkout estimate against the handover update (selected models). The handover update beats the checkout estimate in every window, including the higher-latency windows.

Test results (post hoc). The later test period was already open when the handover stage was designed, so these scores are reported as a post hoc check, not as the basis for selection. Checkout: MAE 5.00, RMSE 6.41, R^2 -0.175 (bias 3.65 days, i.e. the model over-predicts, since the test period delivered faster than training). Handover: MAE 3.97, RMSE 5.44, R^2 0.154 (bias 2.87 days). The handover update’s test R^2 is positive while the checkout estimate’s is negative, so moving the prediction point to handover also makes the estimate more robust to the regime shift between training and test, though it does not remove it.

Same-period benchmark (secondary). A random 5-fold validation and a random 20% test from the saved secondary split give checkout R^2 0.259 (validation) / 0.276 (test, MAE 5.04) and handover R^2 0.386 (validation) / 0.411 (test, MAE 4.34). These line up with public Olist models that use a held-out random split, which suggests the gap between our chronological R^2 and higher published figures is mainly the evaluation design, not the model.

What drives the estimate. coefficient (mean |weight| over windows, numeric and engineered inputs; one-hot state/category dummies excluded from this ranking). At checkout the largest numeric/engineered coefficient is `numeric__promised_days` (1.780), then `logged__distance_km` (1.111) and `numeric__route_typical_days` (0.882); customer-state dummies also carry large weights but are not individually interpretable as a ranked list. At handover, `numeric__handover_days` (3.436) is now the largest numeric/engineered coefficient – the elapsed time itself – ahead of `logged__distance_km` (1.218) and `numeric__promise_slack` (1.003), matching the ladder: once elapsed time is available, it dominates.

Model 2: the late warning (classification)

Method. The classifier outputs the probability that an order misses Olist’s promise, scored the moment the carrier takes the parcel. Candidates follow the same two families as the regression, tuned within each group (Linear: logistic regression over four regularisation strengths, with

and without class weighting; Tree-based: a single tree, a random forest and gradient boosting, each over several depth/leaf settings), then compared with the plain-baseline-first line-up and the same paired one-standard-error rule [3], scored by mean validation PR-AUC. The tuned forest (forest_l50_sqrt) has the highest mean validation PR-AUC (0.369), but tuned logistic regression (logistic_c0.01) is within one standard error (gap 0.002, SE 0.010) and simpler, so it is selected; it also clearly beats its own group’s plain baseline (logistic_c1, PR-AUC 0.343), so the tuning, not just the family choice, earns its place. The product is a ranking: each day operations works down the list from the top. The full candidate table is in Appendix A.

Imbalance. Only 3.5% of test orders are late (4–21% across the five validation windows), so accuracy is meaningless and a fixed 0.5 threshold would flag almost nothing. We report PR-AUC, which is more informative than ROC-AUC when positives are rare [4], against the late rate as the baseline, plus ROC-AUC, precision, recall, F1 and MCC at the chosen operating point.

Operating point. Because risk scores are not stable across periods, operations acts on a *share* of each period’s handovers rather than a fixed probability threshold. The share k is chosen on validation to maximise mean net benefit, valuing a caught late order at 5 times the cost of acting on one order (a net-benefit calculation in the spirit of decision curve analysis [6]; this value is an assumption, not a measured cost). This gives $k = 14\%$.

Table 2: Late warning on the test period, acting on the top 14% of orders by risk.

	PR-AUC	ROC-AUC	Precision	Recall	F1	MCC
Late-warning model	0.23	0.82	11.8%	47%	0.19	0.18
Random list	3.5%	0.50	3.5%	14%	—	0.00

Validation PR-AUC (0.367) is noticeably higher than test PR-AUC (0.23); this tracks the late rate, not a validity problem: per-window PR-AUC rises and falls with the late rate (highest in the Black Friday and Feb–Mar windows, lowest in the calmest ones), and the test period was unusually calm (3.5% late, against 4–21% in validation). ROC-AUC, which is less sensitive to the base rate, is 0.82 on test against 0.769 on validation – a smaller, more expected gap, and evidence that the ranking itself holds up; the net benefit at the validation-chosen 14% share was negative on the calm test period (-0.057 per order), so the list should be sized to the current late rate rather than fixed in advance.

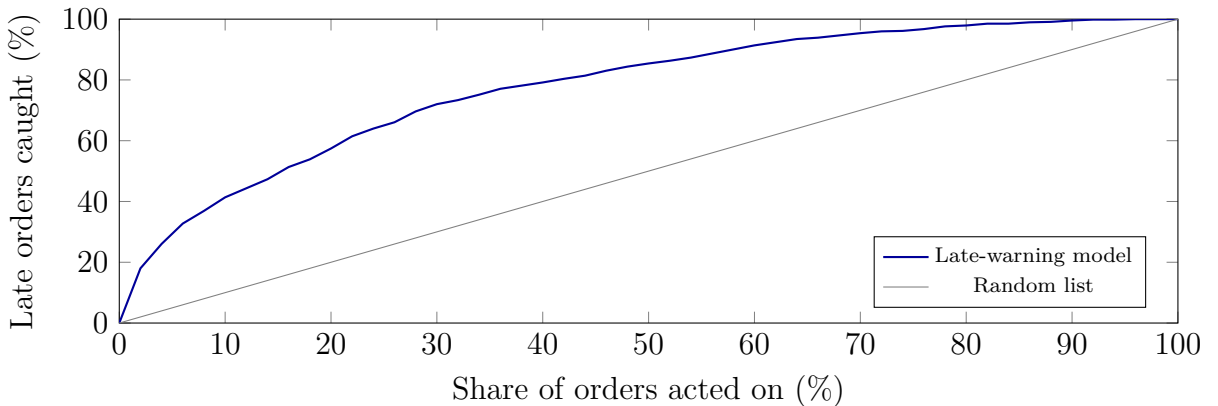


Figure 3: Cumulative share of late orders caught as operations works down the list, test period.

Ranking quality. The top 1%, 5%, 10% and 20% of the daily list contain 13%, 29%, 41% and 57% of the late orders (Figure 3); a random list contains 1%, 5%, 10% and 20%. On validation

the top 10% contained 36% of late orders.

What drives the score. Permutation importance [2] (the drop in validation PR-AUC when a column is shuffled) is led by `handover_days` (0.130), then `remaining_slack` (0.085) and `promised_days` (0.027); all other inputs contribute under 0.02 each. The model is reading mainly how much of the promised time has already elapsed.

Table 3: Monthly test results for the late warning at the 14% share.

Month	Orders	Late orders	Orders acted on	Late orders caught
May (from the 26th)	755	2	106	1
June	6,050	70	847	34
July	6,119	207	857	114
August	6,306	393	883	169

How Olist applies the models

1. **At checkout**, the checkout-stage estimate sets the customer’s expected delivery date.
2. **At handover**, every order gets an updated delivery-time estimate and a risk score; operations works down the day’s ranked list from the top.
3. **For each flagged order**, the updated estimate shows how late it is likely to run: expedite with the carrier if there is still time to recover, otherwise proactively message the customer with a revised date.
4. **Ongoing**, both models are refit and the operating point (k) and candidate line-up are reviewed regularly, because delivery conditions shift (Figure 1) and a stale model or a fixed action share will drift out of step with them.

Limitations and what we would do next

- **Peaks are the hard case.** Both models were trained mostly on calmer months; validation includes the Black Friday and Feb–Mar 2018 windows specifically to test this, and the handover update and the classifier’s ranking lift both hold up there, but with wider errors than in calm periods.
- **Test period was calm.** At 3.5% late, the test period understates how the late-warning model’s net benefit behaves in a high-lateness month; operations should expect the k share and net benefit to need revisiting as conditions change, not treat the test numbers as a steady state.
- **Cohort.** Results describe orders that were eventually delivered with a recorded carrier handover; cancelled and still-open orders have no outcome and are not covered.
- **No causal claim.** We show that delivery time can be estimated and refined, and that late orders can be ranked, not that acting on either output improves customer experience or sales; a pilot with a control group would measure that.
- **A calibrated promise buffer** (turning the checkout estimate into a promise with a chosen on-time rate, deployment detail only) was explored as a stretch goal; it is not part of this report’s headline results.

- **Next step.** Compare the late-warning ranking against the handover estimate’s own implied lateness instead of only Olist’s promise, as a clearly labelled simulation.

A Full configuration tables

Table 4: Every regression candidate, mean over five validation windows.

Stage	Candidate	Family	MAE	RMSE	R^2
Checkout	mean_baseline	mean	7.215	10.671	-0.078
Checkout	linear_baseline	linear	5.860	9.464	0.155
Checkout	ridge_1	ridge	5.858	9.462	0.156
Checkout	ridge_10	ridge	5.848	9.451	0.158
Checkout	ridge_100	ridge	5.809	9.398	0.167
Checkout	tree_baseline	tree	7.787	12.344	-0.472
Checkout	tree_depth6	tree	5.933	9.505	0.144
Checkout	forest_leaf20	forest	5.783	9.340	0.176
Checkout	boost_baseline	boost	5.830	9.452	0.155
Checkout	boost_slow	boost	5.816	9.444	0.157
Handover	mean_baseline	mean	7.215	10.671	-0.078
Handover	elapsed_plus_route_baseline	rule	5.304	8.396	0.337
Handover	linear_baseline	linear	5.146	8.472	0.327
Handover	ridge_1	ridge	5.145	8.470	0.327
Handover	ridge_10	ridge	5.136	8.460	0.329
Handover	ridge_100	ridge	5.099	8.406	0.338
Handover	tree_baseline	tree	6.840	11.625	-0.330
Handover	tree_depth6	tree	5.313	8.633	0.299
Handover	forest_leaf20	forest	5.346	8.808	0.270
Handover	boost_baseline	boost	5.214	8.651	0.297
Handover	boost_slow	boost	5.215	8.666	0.294
Handover	voting_family_winners	voting	5.107	8.471	0.327

Table 5: Every classifier candidate, mean over five validation windows (handover stage).

Candidate	PR-AUC	ROC-AUC	Recall at top 10%
logistic_c1	0.343	0.739	0.349
logistic_c1_bal	0.328	0.734	0.338
logistic_c0.01	0.367	0.769	0.363
logistic_c0.01_bal	0.350	0.761	0.350
logistic_c0.1	0.354	0.755	0.355
logistic_c0.1_bal	0.337	0.743	0.343
logistic_c10	0.338	0.733	0.345
logistic_c10_bal	0.325	0.730	0.335
tree_plain	0.138	0.561	0.213
tree_d4_l20	0.283	0.667	0.287
tree_d4_l100	0.280	0.667	0.286
tree_d6_l20	0.302	0.692	0.313
tree_d6_l100	0.306	0.700	0.321
tree_d8_l20	0.306	0.694	0.297
tree_d8_l100	0.308	0.684	0.322
tree_d12_l20	0.290	0.632	0.294
tree_d12_l100	0.302	0.663	0.309
forest_l5_sqrt	0.360	0.735	0.338
forest_l5_half	0.344	0.723	0.319
forest_l20_sqrt	0.366	0.745	0.341
forest_l20_half	0.354	0.733	0.339
forest_l50_sqrt	0.369	0.747	0.343
forest_l50_half	0.357	0.737	0.344
boost_lr0.03_n200_l20	0.354	0.741	0.333
boost_lr0.03_n200_l100	0.345	0.739	0.329
boost_lr0.03_n500_l20	0.344	0.731	0.320
boost_lr0.03_n500_l100	0.333	0.729	0.318
boost_lr0.1_n200_l20	0.334	0.722	0.312
boost_lr0.1_n200_l100	0.330	0.719	0.309
boost_lr0.1_n500_l20	0.318	0.702	0.299
boost_lr0.1_n500_l100	0.318	0.703	0.300

References

- [1] C. Bergmeir and J. M. Benítez. On the use of cross-validation for time series predictor evaluation. *Information Sciences*, 2012. doi:10.1016/j.ins.2011.12.028.
- [2] L. Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- [3] G. James, D. Witten, T. Hastie and R. Tibshirani. *An Introduction to Statistical Learning*, Sec. 6.1.3, p. 214. Springer, 2013.
- [4] T. Saito and M. Rehmsmeier. The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3):e0118432, 2015.
- [5] N. Salari, S. Liu and Z.-J. M. Shen. Real-time delivery time forecasting and promising in online retailing: when will your package arrive? *Manufacturing & Service Operations Management*, 24(3):1421–1436, 2022.

- [6] A. J. Vickers and E. B. Elkin. Decision curve analysis: a novel method for evaluating prediction models. *Medical Decision Making*, 26(6):565–574, 2006.

Declaration on the use of AI

We used Claude (Anthropic) as a coding and writing assistant for the Phase 2 analysis code, the figures and the first draft of this report. All numbers in this report are generated by `reports/phase2_final_evidence.py` from the saved results in `results/phase2/`. [Team to confirm before submission: that the team specified the design, reviewed the outputs, and takes responsibility for the content.]